ARTIFICIAL INTELLIGENCE • 23A31301T

UNIT IV

First-Order Logic, Inference and Learning

Digital Notes | Diagrams, worked examples and exam tips

SYLLABUS COVERED IN THIS UNIT

First-order logic, inference in FOL, propositional vs first-order inference, unification and lifting, forward chaining, backward chaining, resolution. Learning from observation: inductive learning, decision trees, explanation-based learning, statistical learning methods, reinforcement learning.

1. First-Order Logic (FOL)

First-order logic (also called predicate logic) represents the world as **objects**, **properties** of objects and **relations** between them. It is far more expressive than propositional logic, which has only true/false facts.

Symbol type	Examples	Meaning
Constants	John, Richard, Crown	Names of specific objects
Variables	x, y, z	Stand for any object
Predicates	King(x), Brother(x, y)	Properties and relations (true/false)
Functions	LeftLeg(x), Mother(x)	Map objects to objects
Connectives	¬ ∧ ∨ → ↔	Combine sentences
Quantifiers	∀ (for all), ∃ (there exists)	Make statements about groups of objects
Equality	Mother(John) = Mary	Two terms refer to the same object

Terms, atoms and sentences

- **Term:** an expression that refers to an object: a constant, a variable or a function applied to terms, e.g., *Mother(John)*.
- **Atomic sentence:** a predicate applied to terms, e.g., *Brother(Richard, John)*.
- **Complex sentence:** atomic sentences joined by connectives or quantified, e.g., ∀*x King(x)* ∧ *Greedy(x)* → *Evil(x)*.
- **Ground term / sentence:** contains no variables. A sentence with no free variables is **closed**.

English	FOL
All kings are persons.	∀x King(x) → Person(x)
Some student is clever.	∃x Student(x) ∧ Clever(x)
Every student takes at least one course.	∀x Student(x) → ∃y Course(y) ∧ Takes(x, y)
Brothers are siblings.	∀x, y Brother(x, y) → Sibling(x, y)
No one likes everybody.	¬∃x ∀y Likes(x, y)

REMEMBER
∀ pairs with → and **∃ pairs with ∧**. Writing ∀x King(x) ∧ Person(x) wrongly says *everything* is a king, and ∃x King(x) → Person(x) is trivially true if anything is not a king. The order of mixed quantifiers matters: ∀x ∃y Loves(x, y) (everyone loves someone) is not the same as ∃y ∀x Loves(x, y) (someone is loved by everyone).

2. Inference in First-Order Logic

2.1 Reducing FOL to propositional inference

To use fast propositional methods on FOL knowledge, we first **instantiate** the quantifiers.

Rule	Statement	Example
Universal Instantiation (UI)	From ∀v α, infer α with v replaced by any ground term g	∀x King(x) ∧ Greedy(x) → Evil(x) gives King(John) ∧ Greedy(John) → Evil(John)
Existential Instantiation (EI)	From ∃v α, infer α with v replaced by a **new** constant (Skolem constant) K	∃x Crown(x) ∧ OnHead(x, John) gives Crown(C1) ∧ OnHead(C1, John), where C1 is new

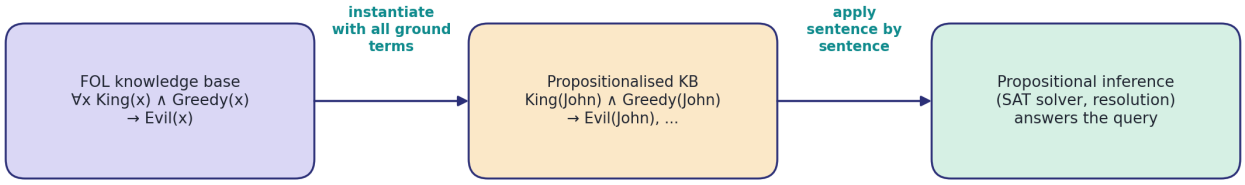


Fig 4.1 Inference by propositionalisation: convert FOL to propositional logic, then reason

Propositionalisation: replace every quantified sentence with all its ground instances, then apply a propositional inference algorithm. It works (Herbrand's theorem: if a sentence is entailed, a finite proof exists), but with function symbols there are infinitely many ground terms, and the method **generates huge numbers of irrelevant sentences**.

	Propositional inference	First-order inference
Expressiveness	Facts only: P, Q, R	Objects, relations, quantifiers
Variables	None	Yes; handled by unification
Knowledge size	Needs a separate sentence for each object	One rule covers all objects
Decidability	Decidable (finite truth tables)	Semi-decidable: if a conclusion is entailed it can be proved, but non-entailment may run forever
Typical method	Truth tables, DPLL / SAT, resolution	Unification + Generalised Modus Ponens, resolution

2.2 Unification and lifting

Instead of generating all instances, **lifted inference** works directly on sentences with variables. It uses **unification** to find substitutions that make different expressions identical.

DEFINITION
Substitution θ = a set of variable bindings such as {x/John, y/Mary}. **Unifier** of two sentences p and q: a substitution θ such that SUBST(θ, p) = SUBST(θ, q). The **most general unifier (MGU)** is the one that places the fewest restrictions on the variables.

Generalised Modus Ponens (GMP) is the lifted version of Modus Ponens. For atomic sentences p_i, p_i', q and a substitution θ with SUBST(θ, p_i') = SUBST(θ, p_i) for all i:

RULE
$$p_1', p_2', \ldots, p_n', (p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow q) \vdash \text{SUBST}(\theta, q)$$

Sentence 1	Sentence 2	Unifier θ
Knows(John, x)	Knows(John, Jane)	{x / Jane}
Knows(John, x)	Knows(y, Bill)	{x / Bill, y / John}
Knows(John, x)	Knows(y, Mother(y))	{y / John, x / Mother(John)}
Knows(John, x)	Knows(x, Elizabeth)	**Fails**: x cannot be John and Elizabeth at once. Fix: *standardise apart* to Knows(x_{17}, Elizabeth), giving {x / Elizabeth, x_{17} / John}
P(x, f(x))	P(a, f(a))	{x / a}

Unification algorithm (idea)

- **1.** If the two expressions are identical, return the empty substitution. If either is a variable, bind it to the other (after the **occur check**).
- **2.** If both are compound (same predicate/function name and arity), unify their arguments **left to right**, applying each new binding to the remaining arguments.
- **3.** Otherwise (different predicates, different constants, arity mismatch) return **failure**.
- **Occur check:** a variable x may not be bound to a term that contains x (e.g., x with f(x) fails), otherwise we would build an infinite term.

3. Forward Chaining

Forward chaining starts from the known **facts** and applies Generalised Modus Ponens to every rule whose premises are satisfied, adding the conclusions as new facts. It stops when the query is derived or no new fact can be generated. It is **data-driven**.

```
function FOL-FC-ASK(KB, query) returns a substitution or false
   repeat until new is empty
       new <- { }
       for each rule (p1 ^ ... ^ pn => q) in KB (standardised apart) do
           for each theta such that SUBST(theta, p1 ^ ... ^ pn) = SUBST(theta, p1' ^ ... ^ pn')
                                    for some facts p1', ..., pn' in KB
               q' <- SUBST(theta, q)
               if q' does not unify with an existing sentence in KB or new then
                   add q' to new;   phi <- UNIFY(q', query)
                   if phi is not fail then return phi
       add new to KB
   return false
```

EXAMPLE

The law says it is a crime for an American to sell weapons to hostile nations. Nono, an enemy of America, has some missiles, and all of them were sold to it by Colonel West, an American. Prove: West is a criminal.

R1: American(x) ∧ Weapon(y) ∧ Sells(x, y, z) ∧ Hostile(z) → Criminal(x)
R2: Missile(x) ∧ Owns(Nono, x) → Sells(West, x, Nono) R3: Missile(x) → Weapon(x) R4: Enemy(x, America) → Hostile(x)
Facts: American(West), Enemy(Nono, America), Owns(Nono, M1), Missile(M1)

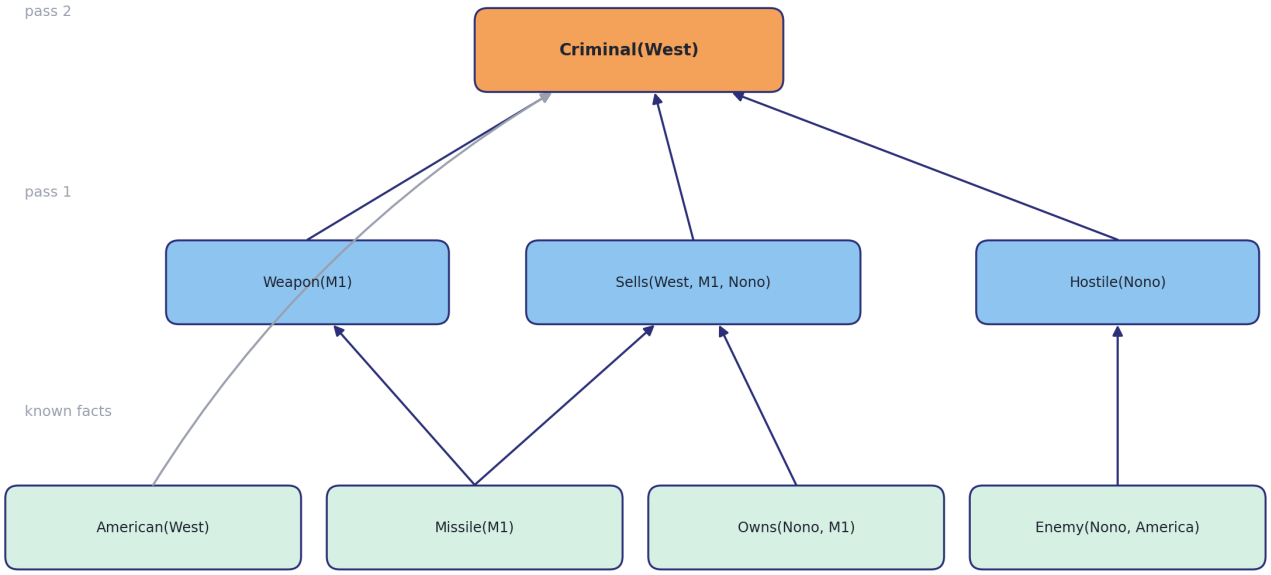


Fig 4.2 Forward chaining proof for 'West is a criminal': new facts are added pass by pass until the query appears

Property	Forward chaining
Sound	Yes: every step is a valid inference
Complete	Yes, for **definite clauses** (at most one positive literal) in a function-free KB (Datalog); terminates in a finite number of steps
Weakness	May derive many irrelevant facts that have nothing to do with the query
Used in	Production systems (OPS5, XCON), deductive databases, monitoring and planning

4. Backward Chaining

Backward chaining starts from the **goal**. It finds rules whose conclusion unifies with the goal and makes their premises the new **sub-goals**, proving them recursively (depth-first, left to right) until all sub-goals match known facts. It is **goal-driven**, the strategy used by Prolog and MYCIN.

```
function FOL-BC-ASK(KB, goals, theta) returns a set of substitutions
   if goals is empty then return { theta }
   q' <- SUBST(theta, FIRST(goals));   answers <- { }
   for each rule (p1 ^ ... ^ pn => q) in KB whose head q unifies with q' using theta':
        new_goals <- [p1, ..., pn | REST(goals)]
        answers <- FOL-BC-ASK(KB, new_goals, COMPOSE(theta', theta)) U answers
   return answers
```

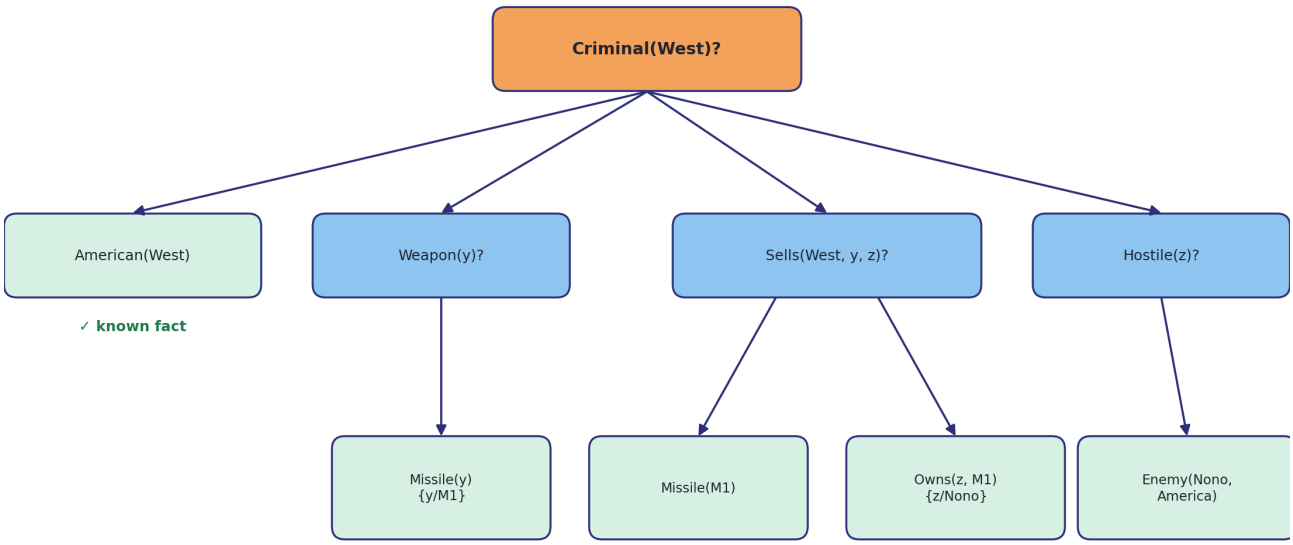


Fig 4.3 Backward chaining: start from the goal, create sub-goals, bind variables, end at known facts

	Forward chaining	Backward chaining
Starts from	Known facts	Query / goal
Efficiency	May produce irrelevant facts	Focused on the goal, so fewer irrelevant steps
Search style	Breadth-style, bottom-up	Depth-first with backtracking, top-down
Risk	Combinatorial explosion of facts	Infinite loops on recursive rules; repeated sub-goals
Typical use	Monitoring, control, configuration	Diagnosis, question answering, Prolog

5. Resolution

Resolution is a single, powerful inference rule that, used with **refutation**, gives a **complete** inference procedure for first-order logic: if the knowledge base entails a sentence, resolution can prove it. It works on sentences in **conjunctive normal form (CNF)**: a conjunction of clauses, where each clause is a disjunction of literals.

RULE
Resolution rule: from $(l_1 \vee \ldots \vee l_k)$ and $(m_1 \vee \ldots \vee m_n)$ where l_i unifies with $\neg m_j$ (unifier θ), infer
SUBST(θ, $l_1 \vee \ldots \vee l_{i-1} \vee l_{i+1} \vee \ldots \vee l_k \vee m_1 \vee \ldots \vee m_{j-1} \vee m_{j+1} \vee \ldots \vee m_n$)
Example: (¬Student(x) ∨ Studies(x)) and Student(Ravi) resolve with {x/Ravi} to give Studies(Ravi).

Converting a sentence to CNF: the six steps

Step	What to do	Example
1	Eliminate ↔ and →: P → Q becomes ¬P ∨ Q	∀x Man(x) → Mortal(x) ⇒ ∀x ¬Man(x) ∨ Mortal(x)
2	Move ¬ inwards (De Morgan, quantifier rules)	¬∀x P ⇒ ∃x ¬P; ¬(P ∧ Q) ⇒ ¬P ∨ ¬Q
3	Standardise variables: give each quantifier its own variable	∀x P(x) ∨ ∃x Q(x) ⇒ ∀x P(x) ∨ ∃y Q(y)
4	**Skolemise**: replace each ∃ variable with a Skolem constant/function of the enclosing ∀ variables	∀x ∃y Loves(x, y) ⇒ ∀x Loves(x, F(x))
5	Drop the universal quantifiers	∀x ¬Man(x) ∨ Mortal(x) ⇒ ¬Man(x) ∨ Mortal(x)
6	Distribute ∨ over ∧ to get clauses	(A ∧ B) ∨ C ⇒ (A ∨ C) ∧ (B ∨ C)

Proof by refutation (contradiction)

To prove a goal α from a knowledge base KB: **(1)** convert KB into clauses; **(2)** add the clauses of **¬α**; **(3)** repeatedly resolve pairs of clauses and add the results; **(4)** if the **empty clause (□)** is produced, KB ∧ ¬α is contradictory, so α is proved.

EXAMPLE

KB: (1) ¬Student(x) ∨ Studies(x) (2) ¬Studies(y) ∨ Passes(y) (3) Student(Ravi). **Goal:** Passes(Ravi). Negate the goal: (4) ¬Passes(Ravi). Resolve (4) with (2), {y/Ravi}: (5) ¬Studies(Ravi). Resolve (5) with (1), {x/Ravi}: (6) ¬Student(Ravi). Resolve (6) with (3): □. Proved.

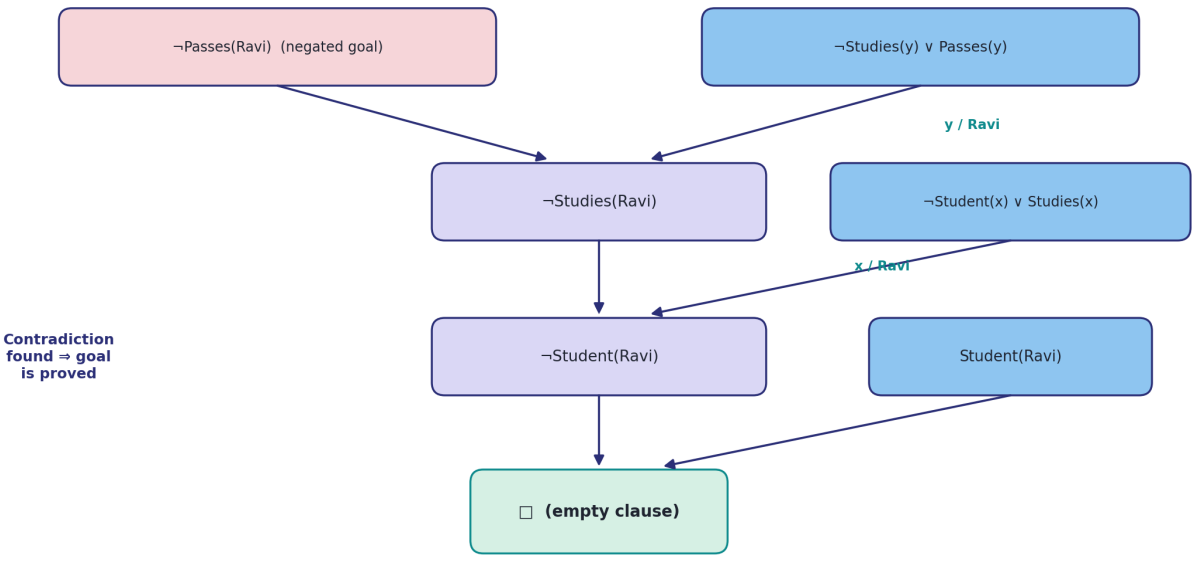


Fig 4.4 Resolution refutation: the negated goal leads to the empty clause, so Passes(Ravi) follows from the KB

- **Sound** (never derives something false) and **refutation-complete** (Gödel/Robinson).
- **Strategies to reduce search:** *unit preference* (resolve with single-literal clauses first), *set of support* (always use a clause from the negated goal or its descendants), *input resolution*, subsumption (delete redundant clauses).

6. Learning from Observation

A **learning agent** improves its performance on future tasks after making observations about the world. Learning is needed when the designer cannot anticipate every situation or when the environment changes.

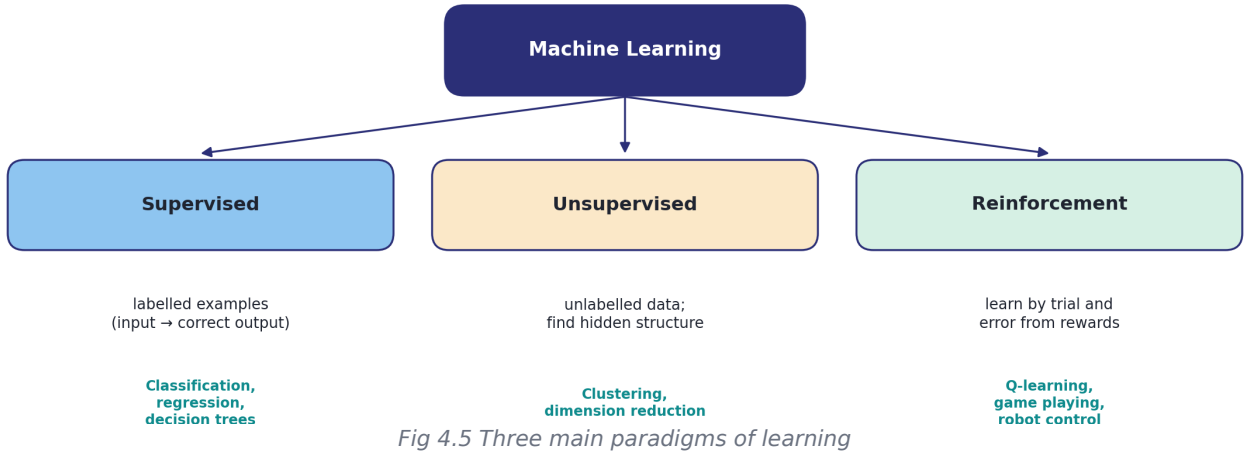


Fig 4.5 Three main paradigms of learning

6.1 Inductive learning

Inductive learning (supervised) learns a function **f** from **examples** (x, f(x)). The goal is a **hypothesis h** that approximates f and **generalises**, i.e., predicts well on new, unseen inputs.

- **Hypothesis space H:** the set of functions the learner can choose from. A hypothesis is **consistent** if it agrees with all training examples.
- **Bias:** any preference for one hypothesis over another. **Ockham's razor:** prefer the simplest hypothesis that is consistent with the data.
- **Overfitting:** a complex h fits noise in the training data and generalises badly. **Underfitting:** h is too simple to capture the pattern.
- **Evaluation:** hold out a **test set**, or use cross-validation; the learning curve shows accuracy as the training set grows.

7. Decision Trees

A **decision tree** takes a vector of attribute values and returns a decision. **Internal nodes** test an attribute, **branches** carry the test outcomes and **leaves** give the class. Trees are easy to read and can be written as IF-THEN rules.

ID3 / decision-tree learning builds the tree top-down. At each step, pick the attribute that best splits the examples, using **information gain** based on **entropy**.

FORMULA

Entropy(S) = $-\,p_+ \log_2 p_+ - p_- \log_2 p_-$ (0 = pure set, 1 = 50/50 mix)

Gain(S, A) = Entropy(S) $-\,\Sigma_{v \in \text{values}(A)}$ ($|S_v| / |S|$) · Entropy(S_v)

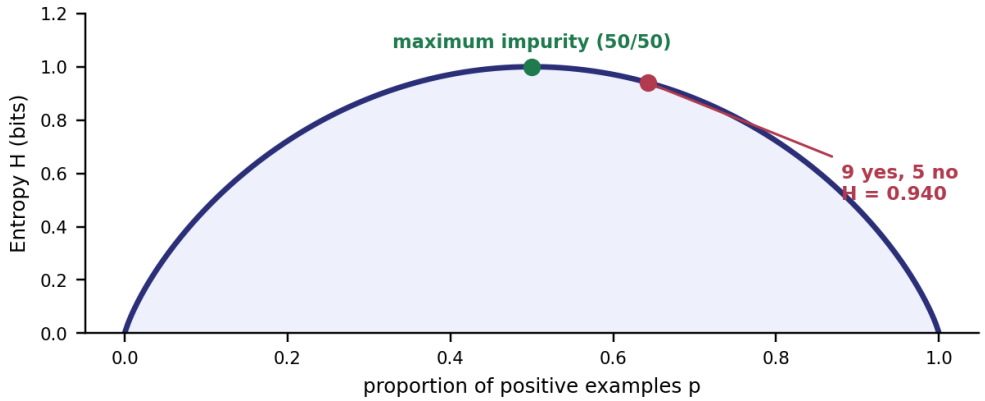


Fig 4.7 Binary entropy: highest when classes are evenly mixed, zero when pure

Worked example: Play Tennis. Decide whether to play tennis (14 days, 9 Yes and 5 No).

Day	Outlook	Temp	Humidity	Wind	Play
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

SOLUTION

Step 1. Entropy(S) = $-(9/14)\log_2(9/14) - (5/14)\log_2(5/14)$ = **0.940**.

Step 2. Gain(Outlook): Sunny (2 Yes, 3 No) = 0.971; Overcast (4 Yes, 0 No) = 0; Rain (3 Yes, 2 No) = 0.971. Gain = 0.940 − (5/14)(0.971) − (4/14)(0) − (5/14)(0.971) = **0.247**.

Similarly Gain(Humidity) = 0.151, Gain(Wind) = 0.048, Gain(Temperature) = 0.029.

Step 3. Outlook has the highest gain, so it becomes the **root**. Overcast is all Yes (a leaf). Repeat for the Sunny and Rain subsets: Sunny splits best on Humidity, Rain on Wind.

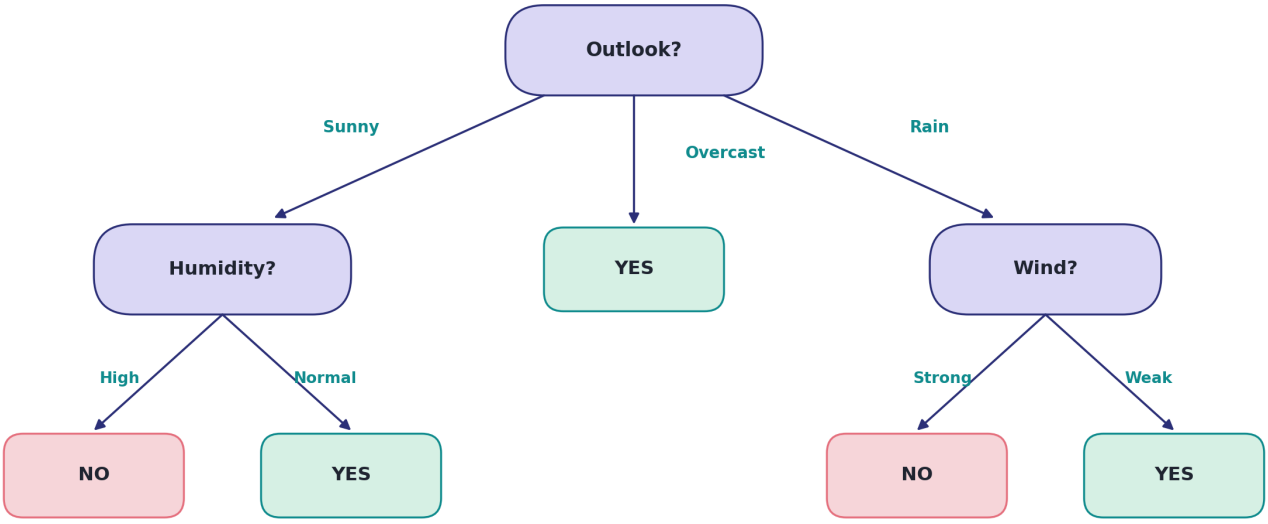


Fig 4.6 Decision tree learned for the 'Play Tennis' data

- **Overfitting control:** *pre-pruning* (stop growing early) and *post-pruning* (cut back subtrees that do not improve validation accuracy).
- **Gain ratio** (C4.5) corrects ID3's bias toward attributes with many values. **Continuous attributes** are handled by choosing a threshold split.

8. Explanation-Based Learning (EBL)

Inductive learning needs many examples. **EBL** learns from a **single** example by using **background knowledge** (a domain theory) to **explain** why the example is an instance of the goal concept, then **generalises** the explanation into a reusable rule.

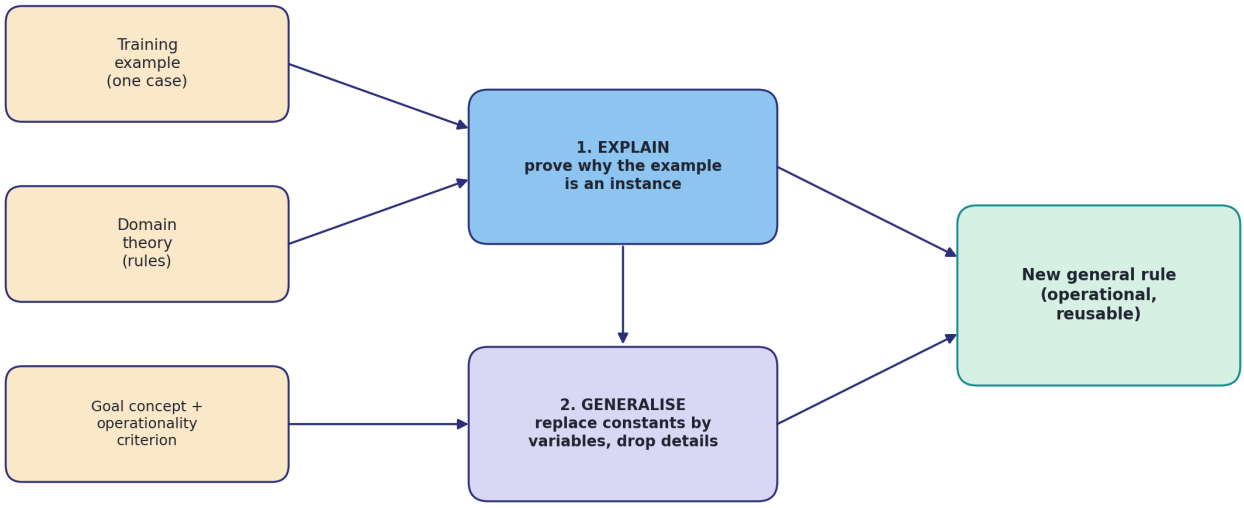


Fig 4.8 Explanation-based learning (EBL): learn a general rule from a single explained example

- **Inputs:** a goal concept, one training example, a domain theory (rules), and an **operationality criterion** (the concept must be expressed with easily checked features).
- **Step 1, Explain:** prove that the example satisfies the goal concept using the domain theory, giving a proof tree. Features not used in the proof are irrelevant.
- **Step 2, Generalise:** replace constants with variables, keeping only the conditions that appear at the leaves of the proof.

EXAMPLE
Concept: Cup. Domain theory: Liftable(x) ∧ HoldsLiquid(x) → Cup(x); Light(x) ∧ HasHandle(x) → Liftable(x); HasConcavity(x) ∧ PointsUp(x) → HoldsLiquid(x).
Example: Obj1 is red, light, has a handle, has an upward-pointing concavity, and is made of plastic. The explanation never uses *red* or *plastic*, so the learned rule is: **Light(x) ∧ HasHandle(x) ∧ HasConcavity(x) ∧ PointsUp(x) → Cup(x)**.

	Inductive learning	Explanation-based learning
Examples needed	Many	One (or few)
Knowledge used	Examples only	Examples + domain theory
Result	New knowledge (may be wrong)	A faster, more efficient form of existing knowledge (always correct if the theory is)
Typical outcome	Hypothesis / decision tree	Operational general rule

9. Statistical Learning Methods

Statistical learning treats learning as **probabilistic inference**, which is useful because data is noisy and uncertain.

Bayesian learning

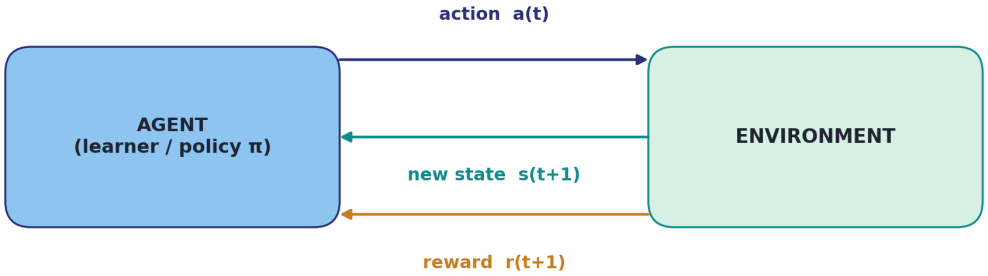
It computes the probability of each hypothesis given the data using Bayes' rule: **P(h | d) = α · P(d | h) · P(h)**. Predictions average over all hypotheses weighted by their posterior probability. Because this is often too costly, two approximations are used:

Method	Choose the hypothesis that…	Formula
MAP (maximum a posteriori)	has the highest posterior probability	h_{MAP} = argmax P(d | h) P(h)
ML (maximum likelihood)	makes the data most likely (assumes all priors are equal)	h_{ML} = argmax P(d | h)

- **Naive Bayes classifier:** assumes attributes are independent given the class, so $P(C \mid x_1 \ldots x_n) \propto P(C) \cdot \Pi\, P(x_i \mid C)$. Very fast, needs little data, used for spam filtering and text classification.
- **Instance-based learning (k-nearest neighbours):** classify a new point by a majority vote of the k closest stored examples. No training phase, but slow at prediction time.
- **Neural networks:** layers of simple units with adjustable weights, trained with back-propagation to reduce the error. They are the basis of deep learning.
- **Support vector machines and EM (clustering with hidden variables)** are other widely used statistical methods.

10. Reinforcement Learning

In **reinforcement learning (RL)** there is no teacher giving correct answers. The agent acts, then receives a numerical **reward** (or punishment), and must learn which actions lead to the highest **long-term** reward. Think of training a dog with treats.



Goal: learn a policy that maximises the total (discounted) future reward

Fig 4.9 The reinforcement-learning loop

Element	Meaning
State s	The agent's current situation
Action a	A choice available to the agent
Reward r	Immediate feedback signal after an action
Policy π(s)	The agent's strategy: which action to take in each state
Utility / value U(s)	Expected total future reward from state s (rewards are discounted by γ, 0 ≤ γ ≤ 1)

Types of reinforcement learning

- **Passive RL:** the policy is fixed; the agent only learns the utility of states by observing, e.g., *adaptive dynamic programming* and *temporal-difference (TD) learning*.
- **Active RL:** the agent must also decide what to do. It faces the **exploration vs exploitation** trade-off: use the best known action (exploit) or try something new (explore).
- **Q-learning** (model-free): learns an action-value function Q(s, a) directly, without needing a model of the environment.

FORMULA

$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$ (α = learning rate, γ = discount factor)

EXAMPLE

Let Q(s, a) = 2, reward r = 1, max Q(s', a') = 5, α = 0.5, γ = 0.9. Target = 1 + 0.9 × 5 = 5.5. New Q = 2 + 0.5 × (5.5 − 2) = **3.75**. The estimate moves toward the target.

- **Applications:** game playing (TD-Gammon, AlphaGo), robot walking and control, self-driving, resource allocation and recommendation.

Quick Revision: Unit IV at a Glance

Topic	One-line recall
FOL	Objects + predicates + quantifiers; ∀ with →, ∃ with ∧
UI / EI	Universal: any ground term. Existential: a new Skolem constant
Unification	Find θ making two expressions identical; MGU is the most general; use the occur check
Forward chaining	Facts to goal (data-driven); complete for definite clauses
Backward chaining	Goal to facts (goal-driven); basis of Prolog
Resolution	CNF + negated goal; derive the empty clause for a proof
Decision tree	Choose the split with the highest information gain
EBL	Explain one example with a domain theory, then generalise
RL	Learn from rewards; Q-learning update with α and γ

Frequently asked exam questions

- Explain propositional vs first-order inference. What are UI and EI?
- Explain unification and lifting with examples. State Generalised Modus Ponens.
- Explain forward and backward chaining with the crime example, and compare them.
- Explain the resolution procedure: CNF conversion steps and a proof by refutation.
- Construct a decision tree for the given data using information gain.
- Write notes on explanation-based learning, statistical learning methods and reinforcement learning.